

Hattrick 复现报告

1. 复现设置

- 数据集: GEANT
- 预测器: ESM
- 路径数: K=8
- 流量类别: High、Medium、Low
- 使用方法: Hattrick、BEST_MC/Flexile、SWAN

2. 没有复现的内容

- 没有对私密数据 PrivateWAN 做复现
- 没有做 UsCarrier / KDL 的大拓扑实验, 预计的训练时间太长
- 没有做消融实验

3. 实验环境

项目	配置
系统环境	Windows 11
Python 版本	3.12.5
PyTorch	2.5.1 + CUDA 12.4
NVIDIA 驱动	610.88

4. 数据处理

GEANT 数据共有 10773 个时间片, 按照论文的方法划分

数据部分	时间片范围	数量
训练集	0-6463	6464
验证集	6464-8079	1616
测试集	8080-10772	2693

最终, 测试集的每个时间片的流量按比例划分为 high, medium, low 三类, 采用三种方法处理, 包含 2693
* 3 * 3 条结果

5. 复现流程

5.1 用 Gurobi 计算基线

生成以下的结果

- ground-truth MaxFlow
- ground-truth MLU
- BEST_MC/Flexile 预测
- SWAN 预测

5.2 Hattrick 训练

使用 README 中的参数训练：

参数	值
topo	geant
num_paths_per_pair	8
epochs	60
batch_size	64
lr	0.0005
num_transformer_layers	3
num_gnn_layers	3
rau1/rau2/rau3	3/3/3
pred	1
pred_type	esm
dynamic	0
violation	1

6. 复现结果

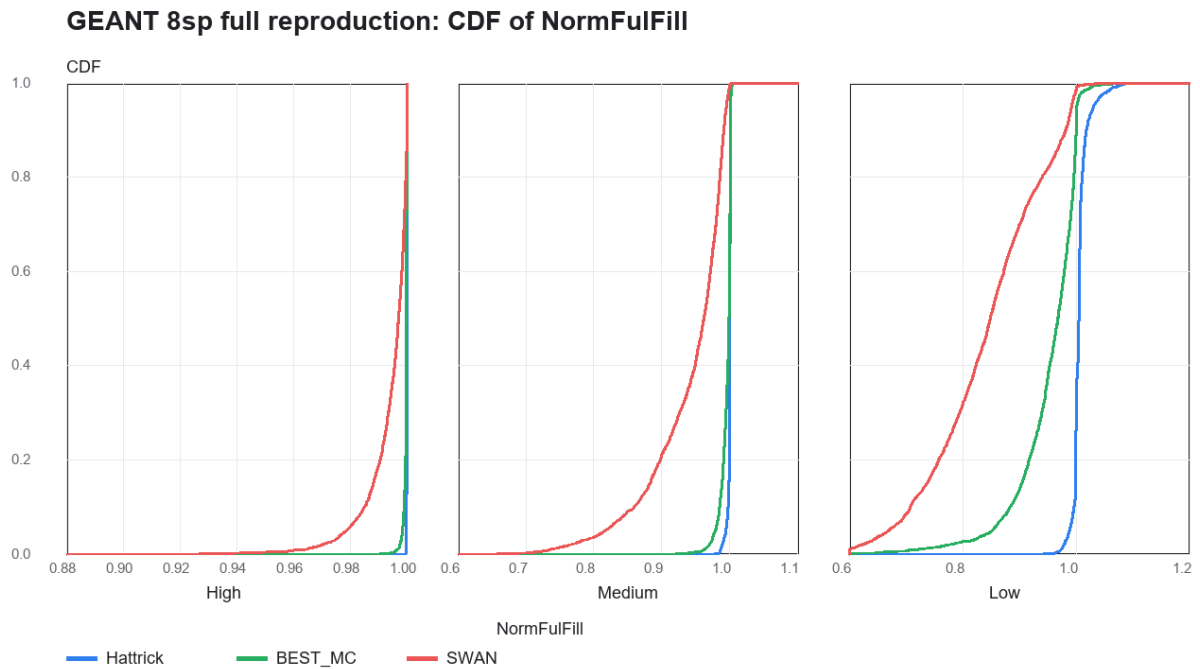
方法	类别	Mean	Median	P10	论文 P10	P10 差值	P1	论文 P1	P1 差值
Hattrick	High	1.000000	1.000000	1.000000	1.000000	-0.000000	1.000000	0.996300	+0.003700
Hattrick	Medium	0.999233	1.000000	0.997324	1.000000	-0.002676	0.986409	0.992100	-0.005691
Hattrick	Low	1.006312	1.005052	0.996565	0.998600	-0.002035	0.974056	0.982400	-0.008344
BEST_MC	High	0.999739	0.999999	0.999073	0.991500	+0.007573	0.996925	0.971000	+0.025925
BEST_MC	Medium	0.994930	0.998599	0.984834	0.965200	+0.019634	0.962555	0.918600	+0.043955
BEST_MC	Low	0.952297	0.968019	0.884473	0.955200	-0.070727	0.722775	0.892400	-0.169625
SWAN	High	0.994322	0.997154	0.985143	0.971300	+0.013843	0.964708	0.944500	+0.020208
SWAN	Medium	0.939935	0.961620	0.859630	0.919400	-0.059770	0.736935	0.832000	-0.095065
SWAN	Low	0.844129	0.848777	0.708226	0.926400	-0.218174	0.598216	0.822900	-0.224684

表格中大于 1 的值表示牺牲了更高优先级流量，使较低优先级类别获得了额外容量

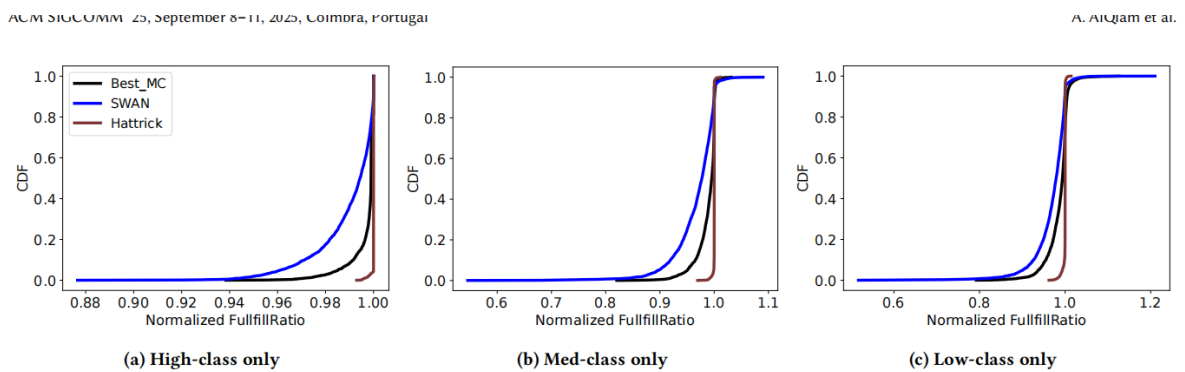
Hatrick 与论文接近，BEST_MC 和 SWAN 的差异相对较大。可能是因为实验中不同优先级流量的比例在某个范围内随机生成，本次实验中 high 比例偏小，更容易满足。BEST_MC 和 SWAN 作为确定性的算法，对这样差别的更敏感，而 Hatrick 通过训练，结果更稳定。

7. 图表结果

7.1 NormFulFill CDF

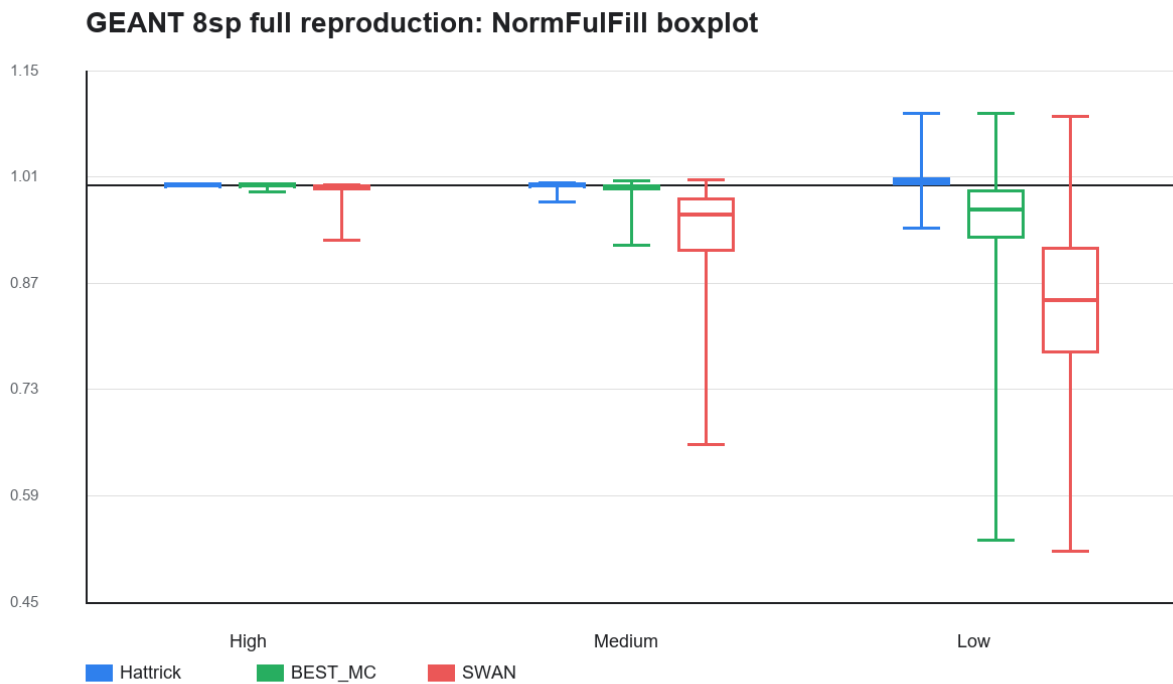


论文中的图像



图像的趋势一致，BEST_MC 和 SWAN 在 high 中偏右，low 和 medium 中偏左，可能是因为 high 的比例偏小

7.2 NormFulFill 箱线图



箱线图显示：

- Hattrick 在 High、Medium 两类上非常稳定
- Hattrick 在 Low 类别仍保持较高分布，但有少量低值
- BEST_MC 和 SWAN 在 Low 类别波动更大

8. 结果对比

Hattrick：

类别	P10 差值	P1 差值	评价
High	-0.000000	+0.003700	基本一致
Medium	-0.002676	-0.005691	非常接近
Low	-0.002035	-0.008344	非常接近

baseline：

- BEST_MC Low 的 P1 比论文低约 0.1696
- SWAN Low 的 P1 比论文低约 0.2247

9. 可能差异原因

1. 流量占比不同，当 high 占比低时，它的 NormFulFill 会更高，medium 和 low 的 NormFulFill 会更低。在基线算法上会很明显，而 Hattrick 在比例发生变化时更加稳定

```
high = rng.uniform(0.45, 0.50, size=matrix.shape)
```

```
mid = rng.uniform(0.40, 0.45, size=matrix.shape)
low = 1.0 - high - mid
```

2. 随机种子，神经网络的随机性等

10. 改进思路

1. 经过测试，不同流量路径集合不同时，Hatrck的效果仍然很好
2. 当链路负载增大时，Hatrck的效果会变差，以下是把数据中所有流量翻倍，链路不变的情况下重新训练后不同算法的表现

方法	流量类别	NormFulFill	实际 FulfillRatio
Hatrck	高	0.9860 / 0.9675 / 0.9735	0.9194 / 0.8599 / 0.8677
Hatrck	中	0.9633 / 0.9374 / 0.9461	0.5936 / 0.4964 / 0.5184
Hatrck	低	1.1610 / 0.9473 / 1.0104	0.5335 / 0.3975 / 0.4383
DOTE-MC default	高	0.9987 / 0.9901 / 0.9965	0.9310 / 0.8796 / 0.8891
DOTE-MC default	中	0.9772 / 0.9178 / 0.9553	0.6028 / 0.4983 / 0.5177
DOTE-MC default	低	1.0780 / 0.9546 / 0.9847	0.5004 / 0.3454 / 0.3804
DOTE-MC sensitivity	高	0.9980 / 0.9891 / 0.9927	0.9303 / 0.8807 / 0.8887
DOTE-MC sensitivity	中	0.9831 / 0.9053 / 0.9673	0.6062 / 0.4812 / 0.5178
DOTE-MC sensitivity	低	1.0711 / 0.9518 / 0.9897	0.4977 / 0.3333 / 0.3562
BEST_MC	高	0.9944 / 0.9824 / 0.9886	0.9270 / 0.8792 / 0.8838
BEST_MC	中	0.9769 / 0.9284 / 0.9534	0.6021 / 0.5066 / 0.5188
BEST_MC	低	0.9256 / 0.6908 / 0.8282	0.4356 / 0.2578 / 0.2935
SWAN	高	0.9921 / 0.9747 / 0.9855	0.9248 / 0.8755 / 0.8814
SWAN	中	0.9269 / 0.8684 / 0.8941	0.5709 / 0.4784 / 0.4940
SWAN	低	0.8872 / 0.6886 / 0.7790	0.4114 / 0.2817 / 0.3040

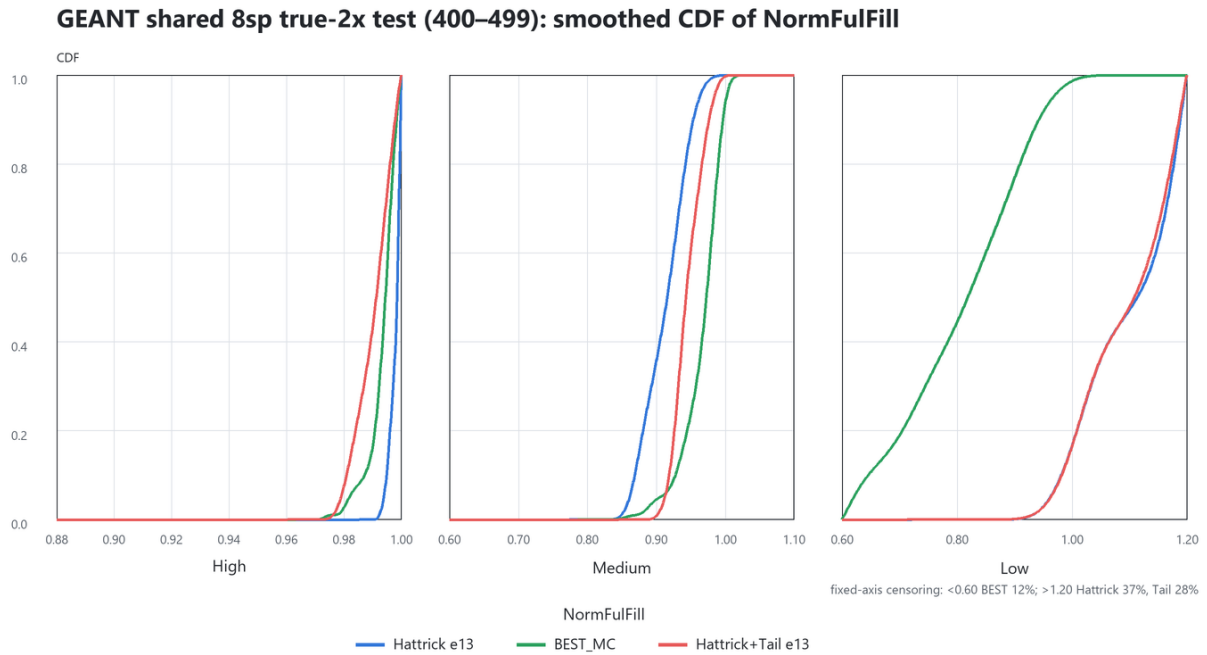
相比于DOTE-MC，Hatrck接收的high和medium流量显著偏低，接受的low过高，已经没有优势。Hatrck系统在高压条件下留给low的资源太多，low无法显示地判断链路压力，在训练刚开始时抢占了过多资源。

在stage1的参数基本稳定后，在残差网络上用BEST_MC求解，效果明显变好

Class	Baseline Mean / P1 / P10	Candidate Mean / P1 / P10
High	0.980723 / 0.952098 / 0.968482	完全相同
Medium	0.841914 / 0.679243 / 0.722949	0.974585 / 0.907224 / 0.942703
Low	1.183867 / 0.966198 / 1.044755	0.729390 / 0.422764 / 0.567846

这里虽然看起来low下降了很多，其实发生的变化只是部分low的流量转移到了medium，流量的总量是接近的。目前的想法是先尝试加一个stage 2/3相关的loss，比如 $\lambda \times (\text{NormFulFill_low} - \text{NormFulFill_mid})$ 。低压条件下也有倒置出现，但很微弱，这个loss不会带来很大的影响。

目前尝试过 $\text{ReLU}(N_1 - N_m)$ ，渐进加入的ReLU，ReLU的二次方，考虑末25%的情况。但是效果都不好。这个loss项并没有把low的流量交给medium，反而像是把high的流量交给了medium



接着会尝试在训练进行到一半时冻结high再引入loss。这个时候high已经调整成合适的路线，让medium+low的流量尽量大。如果效果还是不好，我想先尝试确认一下不同的stage能不能意识到链路压力